

1. What are default and static methods in an interface?

Since **Java 8**, an interface can contain not only abstract methods but also:

1. **Default methods** — instance methods with an implementation.
2. **Static methods** — class/interface-level methods with an implementation.

They solve **different problems**:

Feature	default method	static method
Belongs to	Implementing object	Interface itself
Has implementation	Yes	Yes
Called through	Object/reference	Interface name
Inherited by implementing class	Yes	No
Can be overridden	Yes	No
Can access instance state	Through this	No
Main purpose	Add behavior without breaking implementations	Utility/helper functionality related to interface
Introduced	Java 8	Java 8

2. Why were default methods introduced?

Before Java 8, an interface method was normally abstract:

```
interface Payment {  
    void pay();  
}
```

Suppose you have:

```
class CreditCardPayment implements Payment {  
    @Override  
    public void pay() {  
        System.out.println("Credit Card Payment");  
    }  
}
```

```
class UPIPayment implements Payment {  
    @Override  
    public void pay() {  
        System.out.println("UPI Payment");  
    }  
}
```

Now imagine the interface is already being used by **100 classes**.

If you change the interface:

```
interface Payment {  
    void pay();  
  
    void refund(); // New method  
}
```

all existing implementations must implement:

```
refund();
```

Otherwise they will fail to compile.

This creates a major **backward-compatibility problem**.

Java 8 introduced default methods to solve this problem.

3. Default Method — Definition

A **default method** is an interface method that contains a method body and is declared using the default keyword.

Syntax

```
interface InterfaceName {  
  
    default returnType methodName() {  
        // implementation  
    }  
}
```

Example:

```
interface Payment {  
  
    void pay();  
}
```

```
default void refund() {  
    System.out.println("Default refund processing");  
}  
}
```

Here:

```
void pay();
```

is an abstract method.

Whereas:

```
default void refund() {  
    System.out.println("Default refund processing");  
}
```

is a default method.

4. How a default method works

Consider:

```
interface Payment {  
  
    void pay();  
  
    default void refund() {  
        System.out.println("Refund processed");  
    }  
}
```

Implementation:

```
class CreditCardPayment implements Payment {  
  
    @Override  
    public void pay() {  
        System.out.println("Credit card payment");  
    }  
}
```

Notice that CreditCardPayment does **not** implement refund().

Yet:

```
CreditCardPayment payment = new CreditCardPayment();
```

```
payment.pay();  
payment.refund();
```

Output:

```
Credit card payment  
Refund processed
```

Why?

Because the implementing class automatically receives the default implementation.

Conceptually:

```
Payment interface  
|  
| default refund()  
↓  
CreditCardPayment  
|  
↓  
payment.refund()
```

5. Default method can be overridden

This is one of the most important properties of a default method.

```
interface Payment {  
  
    default void refund() {  
        System.out.println("Generic refund");  
    }  
}
```

Now:

```
class CreditCardPayment implements Payment {  
  
    @Override  
    public void refund() {  
        System.out.println("Credit card refund");  
    }  
}
```

Calling:

```
CreditCardPayment payment = new CreditCardPayment();  
payment.refund();
```

produces:

Credit card refund

So:

Default implementation

↓

Can be inherited

↓

Can be overridden

↓

Subclass/implementation-specific behavior

6. Real-world use case of default methods

Suppose you have:

```
interface Employee {  
    String getName();  
    double getSalary();  
}
```

Many classes implement it:

```
class Developer implements Employee {  
    // ...  
}
```

```
class Manager implements Employee {  
    // ...  
}
```

```
class Tester implements Employee {  
    // ...  
}
```

Now you want to add:

```
displayEmployeeDetails()
```

Without default methods:

```
interface Employee {  
  
    String getName();  
  
    double getSalary();  
  
    void displayEmployeeDetails();  
}
```

Every existing implementation must change.

Instead:

```
interface Employee {  
  
    String getName();  
  
    double getSalary();  
  
    default void displayEmployeeDetails() {  
        System.out.println("Name: " + getName());  
        System.out.println("Salary: " + getSalary());  
    }  
}
```

Now every existing implementation automatically gets this behavior.

This is particularly useful when **evolving public APIs**.

7. Default methods can call abstract methods

This is a very powerful pattern.

```
interface Employee {  
  
    String getName();  
  
    double getSalary();  
  
    default void display() {  
        System.out.println(  
            getName() + " earns " + getSalary()  
        );  
    }  
}
```

The default method doesn't know the actual employee.

Instead, it calls:

```
getName()  
getSalary()
```

which are supplied by the implementing class.

For example:

```
class Developer implements Employee {  
  
    private String name;  
    private double salary;  
  
    Developer(String name, double salary) {  
        this.name = name;  
        this.salary = salary;  
    }  
  
    @Override  
    public String getName() {  
        return name;  
    }  
  
    @Override  
    public double getSalary() {  
        return salary;  
    }  
}
```

Then:

```
Employee e =  
    new Developer("Anurag", 90000);  
  
e.display();
```

The execution is conceptually:

```
e.display()  
↓  
default display()  
↓  
getName()  
↓  
Developer.getName()  
↓  
getSalary()  
↓
```

Developer.getSalary()

This is an example of **template-style behavior**.

8. Default methods are instance methods

A default method is associated with an **object**.

For example:

```
interface Vehicle {  
  
    default void start() {  
        System.out.println("Vehicle starting");  
    }  
}
```

Then:

```
Vehicle v = new Car();
```

```
v.start();
```

You invoke it through an object/reference:

```
v.start();
```

You cannot normally invoke it like:

```
Vehicle.start(); // ✗
```

because start() is not static.

9. Static Method in an Interface

A **static interface method** is a method that belongs to the interface itself rather than to implementing objects.

Syntax

```
interface InterfaceName {  
  
    static returnType methodName() {  
        // implementation  
    }  
}
```

```
}  
}
```

Example:

```
interface Payment {  
  
    static boolean isValidAmount(double amount) {  
        return amount > 0;  
    }  
}
```

Call it using the interface name:

```
boolean result =  
    Payment.isValidAmount(5000);
```

Output:

```
true
```

10. Static interface methods are NOT inherited

This is a very important difference.

```
interface Payment {  
  
    static void process() {  
        System.out.println("Processing payment");  
    }  
}
```

Suppose:

```
class UPIPayment implements Payment {  
}
```

You cannot do:

```
UPIPayment.process(); // ✗
```

You must do:

```
Payment.process(); // ✓
```

Why?

Because the method belongs to:

Payment

not:

UPIPayment

11. Default vs Static — the fundamental difference

Consider:

```
interface Payment {  
  
    default void pay() {  
        System.out.println("Default payment");  
    }  
  
    static void validateAmount(double amount) {  
        System.out.println(  
            amount > 0 ? "Valid" : "Invalid"  
        );  
    }  
}
```

Usage:

```
Payment payment = new UPIPayment();  
  
payment.pay();           // default  
Payment.validateAmount(5000); // static
```

The conceptual model is:

DEFAULT

```
object  
↓  
payment.pay()  
↓  
interface default implementation
```

Whereas:

STATIC

```
interface  
↓
```

```
Payment.validateAmount()  
↓  
static implementation
```

12. When should you use a default method?

Use a **default method** when the behavior logically belongs to **every implementation/object of the interface** and you want to provide a reusable default implementation.

Typical situations:

1. Backward-compatible API evolution

```
interface Customer {  
  
    String getName();  
  
    default String getDisplayName() {  
        return getName();  
    }  
}
```

You can add functionality without forcing every existing implementation to immediately implement it.

2. Common behavior

```
interface Logger {  
  
    default void logInfo(String message) {  
        System.out.println("INFO: " + message);  
    }  
}
```

Every implementation receives the common behavior.

3. Optional behavior

Suppose:

```
interface Report {  
  
    void generate();  
}
```

```
default void exportToPDF() {  
    System.out.println("Default PDF export");  
}  
}
```

An implementation can accept the default or customize it.

4. Framework/library API evolution

This is one of the biggest real-world reasons.

Imagine a library has:

```
interface DataProcessor {  
  
    void process();  
}
```

Thousands of applications implement it.

A library developer wants to add:

```
default void validate() {  
    System.out.println("Default validation");  
}
```

Existing implementations don't necessarily have to change.

13. When should you use a static interface method?

Use a **static interface method** when the functionality:

- logically belongs to the interface/domain,
- does not require object state,
- should not be overridden,
- should not be inherited by implementations,
- is useful as a helper/utility operation.

Example:

```
interface Employee {  
  
    static boolean isValidSalary(double salary) {  
        return salary >= 0;  
    }  
}
```

```
}  
}
```

Call:

```
if (Employee.isValidSalary(50000)) {  
    System.out.println("Valid salary");  
}
```

There is no need to create:

Employee employee = ...

because validation does not depend on an employee object.

14. Real-time example — Payment System

Consider a banking/payment application.

```
interface Payment {  
  
    void pay(double amount);  
  
    default void printReceipt(double amount) {  
        System.out.println(  
            "Payment receipt: ₹" + amount  
        );  
    }  
  
    static boolean isValidAmount(double amount) {  
        return amount > 0;  
    }  
}
```

Implementation:

```
class UPIPayment implements Payment {  
  
    @Override  
    public void pay(double amount) {  
  
        if (!Payment.isValidAmount(amount)) {  
            System.out.println("Invalid amount");  
            return;  
        }  
  
        System.out.println(  

```

```
        "UPI payment processed: ₹" + amount
    );
}
}
```

Usage:

```
Payment payment = new UPIPayment();
payment.pay(5000);
payment.printReceipt(5000);
```

Here:

Static method

```
Payment.isValidAmount(5000);
```

is a utility/helper operation.

Default method

```
payment.printReceipt(5000);
```

is object-related behavior.

15. Real-time example — Java Collections

Default methods became extremely important in Java's collection APIs.

For example, `java.util.Collection` and related interfaces gained default methods in Java 8.

A famous example is:

```
Iterable.forEach()
```

Conceptually:

```
interface Iterable<T> {
    default void forEach(
        Consumer<? super T> action) {
        for (T t : this) {
            action.accept(t);
        }
    }
}
```

```
}  
}  
}
```

This allows implementations of Iterable to obtain common iteration behavior.

Usage:

```
List<String> names =  
    Arrays.asList("A", "B", "C");  
  
names.forEach(  
    name -> System.out.println(name)  
);
```

The important idea is:

```
Iterable  
↓  
default forEach()  
↓  
List implementation  
↓  
names.forEach(...)
```

This is a real-world example of **default methods providing reusable behavior across many implementations**.

16. Another real-world example — Comparator

Comparator is another important Java interface that uses default and static methods.

For example:

```
Comparator<String> byLength =  
    Comparator.comparingInt(String::length);
```

Comparator provides static factory/helper methods such as:

```
Comparator.comparing(...)  
Comparator.comparingInt(...)  
Comparator.naturalOrder()  
Comparator.reverseOrder()
```

and default methods such as:

```
thenComparing(...)
reversed()
```

For example:

```
Comparator<Employee> comparator =
    Comparator.comparing(Employee::getName)
                .thenComparing(Employee::getSalary);
```

Here:

```
Comparator.comparing(...)
```

is a **static method**.

It creates/builds a comparator.

Whereas:

```
.thenComparing(...)
```

is **default/instance behavior** on an existing comparator.

Conceptually:

```
Comparator.comparing(...)
    ↓
creates Comparator
    ↓
.thenComparing(...)
    ↓
creates combined behavior
```

This is an excellent real-world example of why Java provides both mechanisms.

17. Default method and multiple inheritance problem

Java allows a class to implement multiple interfaces:

```
interface A {

    default void show() {
        System.out.println("A");
    }
}
```

```
interface B {  
  
    default void show() {  
        System.out.println("B");  
    }  
}
```

Now:

```
class Test implements A, B {  
}
```

This creates a conflict.

Which show() should be used?

```
A.show()  
  \  
   → Test.show() ???  
  /  
B.show()
```

Java forces the class to resolve the conflict.

```
class Test implements A, B {  
  
    @Override  
    public void show() {  
        System.out.println("Test");  
    }  
}
```

Now:

```
Test t = new Test();
```

```
t.show();
```

Output:

```
Test
```

18. Calling a specific interface's default method

Java provides:

```
InterfaceName.super.methodName();
```

Example:

```
interface A {  
  
    default void show() {  
        System.out.println("A");  
    }  
}
```

```
interface B {  
  
    default void show() {  
        System.out.println("B");  
    }  
}
```

Implementation:

```
class Test implements A, B {  
  
    @Override  
    public void show() {  
  
        A.super.show();  
        B.super.show();  
  
        System.out.println("Test");  
    }  
}
```

Output:

```
A  
B  
Test
```

This allows the implementing class to explicitly select interface default behavior.

19. Important restriction — interface static methods

Static methods cannot be overridden.

For example:

```
interface A {  
  
    static void show() {
```

```
    System.out.println("A");
}
}
```

Then:

```
class Test implements A {
}
```

This does **not** mean:

```
Test.show(); // ✗
```

Instead:

```
A.show(); // ✓
```

And an implementation cannot override the interface static method.

20. Default vs abstract vs static methods

An interface can contain all three:

```
interface Employee {

    // Abstract
    String getName();

    // Default
    default void printName() {
        System.out.println(getName());
    }

    // Static
    static boolean isValidName(String name) {
        return name != null &&
            !name.isBlank();
    }
}
```

These have three different purposes:

Abstract

```
String getName();
```

Means:

Every implementation must provide this behavior.

Default

```
default void printName() {  
    System.out.println(getName());  
}
```

Means:

I provide a reusable implementation, but an implementation class may customize it.

Static

```
static boolean isValidName(String name) {  
    return name != null && !name.isBlank();  
}
```

Means:

This operation belongs to the interface itself and does not require an implementing object.

21. A practical decision algorithm

When designing an interface, ask these questions.

Question 1

Does every implementation need to provide its own implementation?

Use:

abstract method

Example:

```
void pay();
```

Question 2

Can I provide one common implementation that most/all implementations can use?

Use:

default method

Example:

```
default void printReceipt() {  
    ...  
}
```

Question 3

Does the operation need an object/instance?

If **yes**, consider:

default/abstract instance method

If **no**, consider:

static method

Question 4

Should implementing classes inherit the behavior?

If yes:

default

If no:

static

Question 5

Should implementations be able to customize the behavior?

If yes:

default

If no:

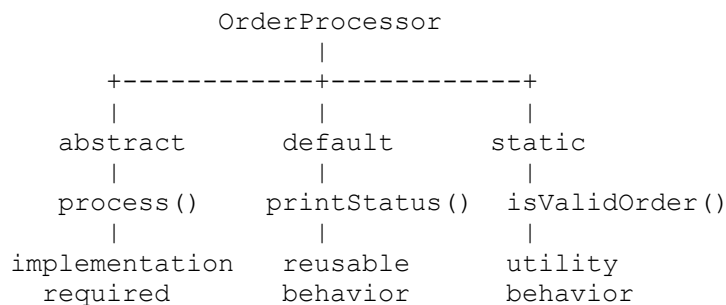
static

22. Real-world design pattern

Consider an order-processing system:

```
interface OrderProcessor {  
  
    // Every processor must implement this  
    void process(Order order);  
  
    // Common behavior  
    default void printStatus(Order order) {  
        System.out.println(  
            "Order status: " + order.getStatus()  
        );  
    }  
  
    // Utility/helper  
    static boolean isValidOrder(Order order) {  
        return order != null;  
    }  
}
```

The three methods have different responsibilities:



This separation is extremely useful when designing enterprise APIs.

23. Common mistakes

Mistake 1 — Treating default as static

Wrong:

```
Payment.refund();
```

if `refund()` is a default method.

Correct:

```
Payment payment = new UPIPayment();  
payment.refund();
```

Mistake 2 — Calling static through an object

Avoid:

```
Payment payment = new UPIPayment();  
payment.isValidAmount(5000); // ✗
```

Correct:

```
Payment.isValidAmount(5000);
```

Mistake 3 — Trying to override a static interface method

Static methods belong to the interface.

```
interface Payment {  
    static void validate() {  
    }  
}
```

It should be called:

```
Payment.validate();
```

It isn't polymorphic instance behavior.

24. Key difference in one example

```
interface Vehicle {  
    default void start() {  
        System.out.println("Vehicle starts");  
    }  
  
    static boolean isValidSpeed(int speed) {  
        return speed >= 0;  
    }  
}
```

```
}  
}
```

Usage:

```
Vehicle vehicle = new Car();
```

```
vehicle.start();
```

because:

```
start()
```

↓

instance behavior

↓

default method

But:

```
Vehicle.isValidSpeed(80);
```

because:

```
isValidSpeed()
```

↓

interface-level utility

↓

static method

25. Summary table

Characteristic	Abstract	Default	Static
Method body	✗	✓	✓
Java 8 feature	No	Yes	Yes
Instance method	Yes	Yes	No
Belongs to object	Yes	Yes	No
Belongs to interface	Declaration	Implementation inherited	Yes
Automatically inherited	N/A	Yes	No
Can override	Must implement	Yes	No

Called as obj.method()	Yes	Yes	No
Called as Interface.method()	No	No	Yes
Uses object state	Yes	Yes	No
Main purpose	Contract	Reusable default behavior	Utility/helper
Polymorphic	Yes	Yes	No

The easiest way to remember

Think of an interface as a **contract + reusable behavior + utilities**:

INTERFACE

- |
 - ├─ abstract method
 - └─ "You MUST implement this."
 - ├─ default method
 - └─ "I provide a default implementation;
you MAY override it."
 - └─ static method
 - └─ "This utility belongs to the interface;
call it using InterfaceName."

Practical rule

Use abstract when the implementation must be supplied by each class.

Use default when the interface should provide common instance behavior that implementations may override.

Use static when you need an interface-related utility/helper that does not depend on an object and should not participate in polymorphism.

A particularly useful real-world distinction is:

Comparator.comparing(...)

→ **static factory/helper behavior**

versus

`comparator.thenComparing(...)`

→ **instance/default behavior**

That distinction is a good mental model for understanding why Java has both default and static methods in interfaces.